

软件工程经济学视角下的产出物资产化治理：从维护负债到可信资产

Artifact Assetization Governance from the Perspective of Software Engineering Economics: From Maintenance Liability to Trusted Assets

作者：付毅

Author: FU Yi

作者单位：川北医学院 后勤管理处，四川 南充 637000

Affiliation: Logistics Management Office, North Sichuan Medical College, Nanchong, Sichuan 637000, China

关键词：软件工程经济学；软件资产化；封板机制；维护义务契约；产出物治理；信创战略；存在性负债

Keywords: software engineering economics; software assetization; sealing mechanism; maintenance obligation contract; artifact governance; Xinchuang strategy; existential maintenance liability

术语速查表

术语	英文	简要定义
存在性负债 (EML)	Existential Maintenance Liability	代码部署后因持续维护义务而形成的隐性经济负担
封板	Sealing	通过技术与管理手段锁定产出物状态，将维护义务从"连续无限"转为"资产单元级、可审计"
解封	Unsealing	对已封板资产进行修改，使其重新进入流变状态并恢复维护负债
再封板	Re-sealing	变更完成后重新执行封板流程，闭合治理环
SBOM	Software Bill of Materials	软件物料清单，记录产出物全部依赖及来源
SLSA	Supply-chain Levels for Software Artifacts	谷歌主导的软件供应链安全成熟度框架

术语	英文	简要定义
MTTR	Mean Time To Recovery	平均故障恢复时间
RTO	Recovery Time Objective	业务可容忍的最大恢复时间目标

摘要

软件维护成本在大型传统行业中常占IT预算70%以上，凸显传统"代码即资产"认知的局限。本文提出基于"维护义务契约"的治理框架，将源代码视为承载了沉重维护义务的"存在性负债"（EML）。不同于传统视代码为静态资产的视角，本文认为：未经治理的代码，其持续维护成本往往掩盖了其业务价值，呈现出显著的负债特征。核心机制"封板"通过标准化操作锁定责任边界，将流变代码转化为功能确定、可度量的产出物资产。创新点在于：1) 提炼EML概念，与技术债范式形成互补；2) 构建"封板强度谱系"及"解封-再封板"动态闭环；3) 结合人员高流动与信创需求，提供本土化实施路径。通过银行核心系统改造的复合示范案例，系统展示框架应用，为构建可信软件资产管理体系提供兼具理论深度与操作性的解决方案。

Abstract

Software maintenance costs often exceed 70% of IT budgets in large traditional industries, highlighting the limitations of the conventional "code as asset" paradigm. This paper proposes a governance framework based on the "Maintenance Obligation Contract," treating source code as "Existential Maintenance Liability" (EML) that carries significant maintenance obligations. Unlike the traditional view of code as a static asset, this paper argues that ungoverned code, with its ongoing maintenance costs, often overshadows its business value and exhibits significant liability characteristics.

The core mechanism, "Sealing," locks responsibility boundaries through standardized operations, transforming fluid code into functionally deterministic and measurable artifact assets. The sealing mechanism is distinguished from conventional release practices: while release is an outward-facing delivery action, sealing is an inward-facing governance operation that explicitly freezes maintenance responsibilities and cost boundaries. This paper introduces a "sealing intensity spectrum" ranging from strong sealing (e.g., signed container images with SBOM) to weak sealing (e.g., CI-passed Git commits), enabling organizations to balance asset determinism against change agility.

Furthermore, this framework incorporates dynamic governance through "unsealing" and "re-sealing" mechanisms. When a sealed asset requires modification, it triggers an unsealing event that restores the asset to a fluid state and reactivates maintenance liabilities. To prevent governance degradation, clear unsealing thresholds and re-sealing deadlines are established. A multi-dimensional risk

assessment framework is also proposed, evaluating code modules across cognitive load, change frequency, dependency stability, responsibility clarity, and failure history dimensions.

The innovations include: 1) refining the EML concept as a complement to the technical debt paradigm; 2) constructing a "sealing intensity spectrum" and "unsealing-resealing" dynamic closed-loop; 3) providing localized implementation paths tailored to China's high personnel mobility and Xinchuang (indigenous innovation) strategy requirements. Through a composite demonstration case of bank core system transformation, this paper systematically demonstrates the framework's application, offering a solution with both theoretical depth and operational feasibility for building trusted software asset management systems.

一、引言

国内金融、电信等行业核心系统动辄承载千万行遗留代码，年度维护成本可达初始投资数倍[1]，且因代码缺陷引发的业务中断风险时有发生[2]。传统技术债理论聚焦"决策性债务"，却难以解释为何结构优良的代码同样随系统老化而沦为维护重负。

贯穿案例：某全国性股份制银行核心的COBOL"利息计算模块"（约50万行代码），作为影响千万级客户的核心组件，其唯一精通专家临近退休，文档缺失。据统计，其单次功能适配平均需3人月，年度隐性维护成本（含人力、故障处置、合规适配）超百万元，已成为典型"负资产"。

本文认为，软件的价值实现于其可预期的、可靠的行为，而非静态的代码文本。基于在多个大型组织中的实践观察，我们总结出 维护义务契约 的分析框架：源代码因承载组织关于长期维护的隐性契约而成为"存在性负债"；软件工程的核心经济活动，是通过"封板"机制，系统性将此负债转化为可验证、可流通的"产出物资产"。本文的核心论点并非否定代码的资产属性，而是指出：未经有效治理（封板）的代码，其隐性的、持续增长的维护负债往往吞噬了其作为资产的价值。

本文核心贡献：

- 概念提炼：**提炼"存在性负债"（EML）概念，明确其与技术债的范式差异。
- 机制总结：**总结"封板"强度谱系，并引入"解封"与"再封板"概念，形成可落地的动态治理闭环。
- 本土适配：**紧扣中国人员高流动性市场特征与信创战略，阐明"产出物资产化"的实施路径。

二、理论基础

2.1 核心经验命题与理论推演

本框架建立在三个广泛可观察的软件工程经验命题之上：

命题一（流变性）：处于活跃维护状态的源代码，其功能、依赖与环境通常随时间持续演进（流变）。

命题二（维护义务）：组织对已部署并产生业务价值的代码，负有确保其持续正确运行的默示维护义务。

命题三（成本伴生）：履行维护义务需持续投入经济资源（人力、计算、管理成本）。

由此可形成本文核心推论：

- 存在性负债：**由命题一与命题二，流变的代码往往绑定未来持续的维护义务；由命题三，该义务通常意味着确定的未来经济成本流出。因此，持续演进的代码在经济学意义上可视为一项**"存在性负债"**（**Existential Maintenance Liability, EML**）。
- 封板的价值：**若要终止此项负债的无限累积，最直接且可审计的途径是终止其流变性（打破命题一）。这需要一种机制，将代码从"流变"状态转化为"确定"状态。本文称其为**"封板"**（**Sealing**）。
- 动态治理的闭环：**封板并非一劳永逸。当外部需求或环境迫使已封板的资产必须修改时（即"解封"），该资产即重新回归"流变"状态，维护义务与成本负债也随之恢复（再负债化，即资产重新承载维护义务）。

2.2 存在性负债与技术债：范式比较与演进

技术债理论[3]开创了用经济隐喻分析软件质量的先河。软件维护成本的量化与管理，长期以来是软件经济学（Boehm & Papaccio, 1988）[12]的核心关切。近期研究（Ampatzoglou et al., 2019）[13]也愈发关注技术债的金融属性。本文提出的EML实现了视角转换，两者在以下维度存在本质差异：

表1 存在性负债（EML）与技术债的范式比较

Table 1 Paradigm comparison between EML and technical debt

维度	技术债 (Technical Debt)	存在性负债 (EML)
本质	为换取速度而做出的技术妥协	软件存在所固有的维护义务契约
产生时点	决策时点明确（如为赶工写坏代码）	存在即产生，随代码部署而默示生效
偿还方式	重构/重写（改善代码结构）	封板（锁定状态，转型维护义务形态）
计量核心	利息（随时间增加的额外维护成本）	本金现值（未来维护义务的预期总成本折现）

维度	技术债 (Technical Debt)	存在性负债 (EML)
管理目标	优化代码质量，降低长期成本	明晰责任边界，将不确定义务转化为可控资产

技术债关注"如何把代码写得更好"，而EML关注"如何让写好的代码不再成为组织的负担"。后者是前者的治理升维。

2.3 与既有工程概念的谱系定位

"封板"在技术操作层借鉴了"基线"、"不可变基础设施"等思想，但其独特价值在于治理意图的升维：

- **基线 (Baseline)**: 是配置管理概念，目标为"版本控制与回溯"。
- **发布 (Release)**: 是交付流程概念，目标为"将新功能提供给用户"。
- **不可变基础设施 (Immutable Infrastructure)**: 是运维范式，目标为提升环境一致性、简化部署与回滚。
- **封板 (Sealing)**: 是治理与经济学概念，其目标是通过技术上的"不可变"来实现 "维护责任的显式冻结" 与 "成本边界的清晰划定"。简言之，"不可变"是封板的技术手段，而"责任与成本锁定"是封板的管理目的。

"封板"是将传统的工程控制点，转化为经济责任的结算点。

2.4 管理视角下的"类负债义务"属性

软件"存在性负债"在管理实质上可类比于会计准则中的"预计负债"[4]，它是由过去开发行为形成的现时义务，导致未来经济利益很可能流出。这为在企业管理层面建立"技术负债风险附录"提供了分析框架。

需要强调的是，本文使用的"存在性负债"及"类负债义务"等术语，均用于管理会计（**Management Accounting**）与内部治理分析范畴，旨在提供风险量化与成本管理的决策框架。它们不构成《企业会计准则》下需在财务报表中确认、计量与披露的法定负债（**Financial Liability**）依据。¹

2.5 信创战略与供应链经济学下的软件资产可信性

在软件供应链高度复杂与开源"公地悲剧"[5]并存的今天，SBOM（软件物料清单）[6]、SLSA框架[7]等实践推动了资产可信化。"封板"实质是构建"可验证产权边界"，以降低供应链中的信任成本。

中国"信创"（信息技术应用创新）战略从国家层面强化了软件供应链自主可控要求[11]。企业的"封板"实践（签名、SBOM），正是对接国家战略、将外部合规压力转化为内部治理升级的关键桥梁。

三、 核心框架：存在性负债与风险评估

3.1 "存在性负债"与"维护义务契约"

定义：软件源代码的"存在性负债"（Existential Maintenance Liability, EML），指源于组织默认或明示的"长期维护义务契约"，并为履行该契约而预期需持续投入经济资源的现时义务。

概念图解：下图勾勒了从代码到资产的动态管理闭环（图1）：

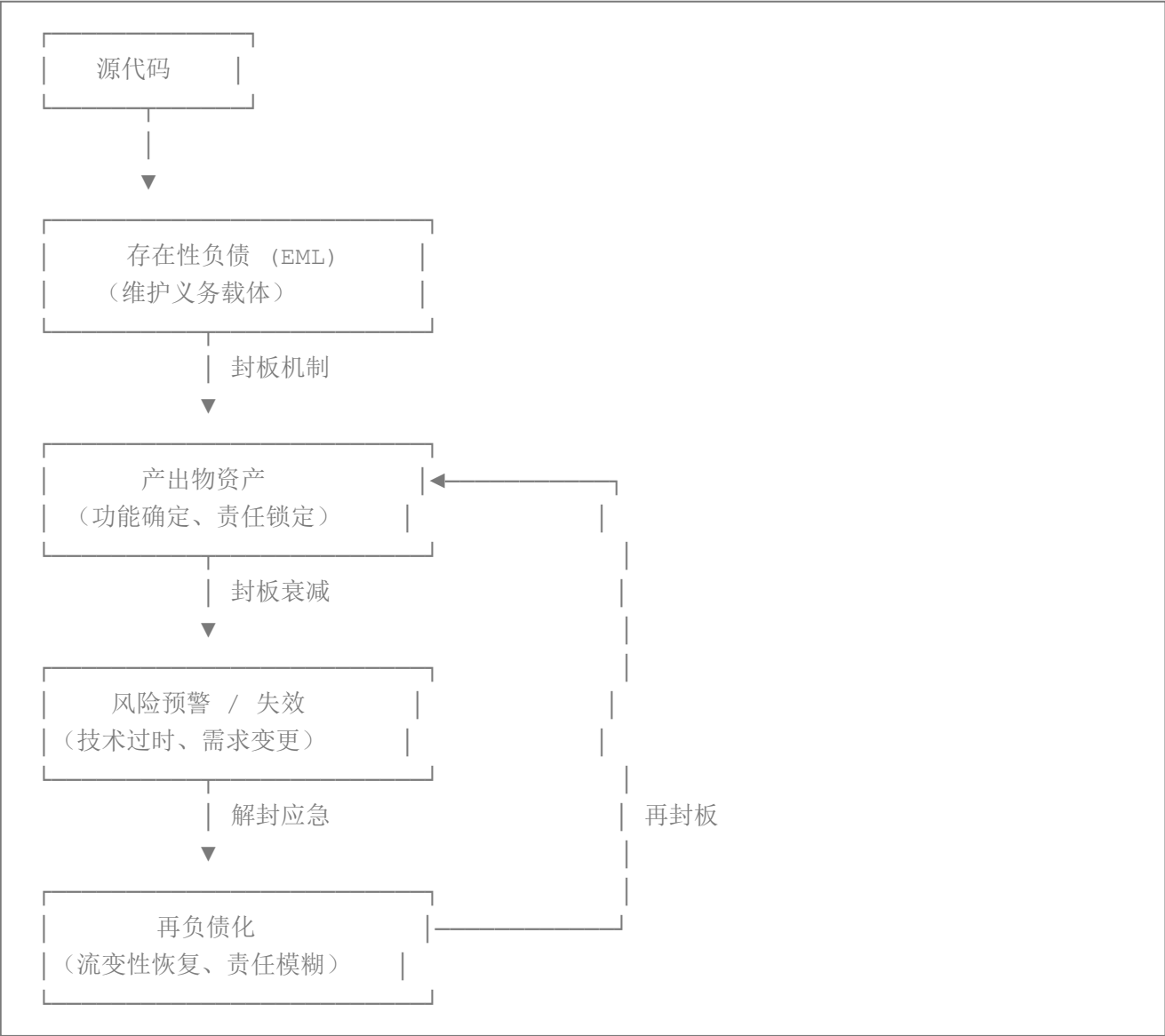


图1 存在性负债与封板治理闭环

Fig.1 Existential maintenance liability and sealing governance closed-loop

"维护义务契约"的实践意涵：此概念植根于工程实践中的"运维责任"，在软件语境下特指组织对已部署代码持续的、默示的照管责任。

3.2 存在性负债的多维风险评估框架

说明：下表为风险等级评估工具，其输出用于内部管理优先级排序，而非财务会计计量。

表2 代码"存在性负债"风险评估框架
Table 2 Risk assessment framework for code EML

维度	低风险 (1分)	中风险 (2分)	高风险 (3分)	高流动性市场影响	供应链安全要求
认知负荷	模块独立，文档完整。	耦合增加，文档部分过时。	高度耦合，文档缺失，逻辑晦涩。	权重高：知识易流失	常规
变更频率	年均<1次。	季度级变更。	月度或更频繁。	常规	常规
依赖稳定性	依赖为长期支持版本。	依赖定期更新但可控。	依赖快速迭代或近EOL。	常规	权重高：替代来源少
责任明晰度	有明确维护团队。	责任共享。	无人负责或关键人将离职。	权重高：权责需清晰	常规
故障历史	几乎无故障。	偶发故障。	故障频发或影响重大。	常规	权重高：影响连续性

案例评分：银行COBOL模块在认知负荷（专家退休）、责任明晰度（无备份）等高风险维度得分极高，综合风险等级为"高"，应优先启动资产化改造。

四、 价值转化：封板机制、衰减与闭环

4.1 "封板"：责任转型与资产创造

"封板"（Sealing）是一组标准化、可验证的技术与管理操作，旨在将背负维护义务的流变代码，转化为一个状态确定、责任锁定的产出物。

本文所称"资产"，均指管理与治理意义上的可控产出物单元，而非严格意义上的财务会计确认资产。

需要特别澄清的是：封板并非"消灭"维护义务，而是"转型"维护义务的形态——从连续、无限、隐性的责任，转变为资产单元级、有限、可审计的责任。封板后的资产仍需监控与维护，但这种维护是显性的、有边界的、可计量的。

4.1.1 封板 ≠ 发布：责任状态的本质差异

表3 封板与发布的本质区别

Table 3 Essential differences between sealing and release

维度	发布（Release）	封板（Sealing）
关注焦点	代码状态变更	责任状态变更
维护义务形态	连续、无限、隐性（默认延续）	资产单元级、有限、可审计（需经解封门 禁方可修改）
产出物属性	工程中间产物	可治理的资产单元
管理视角	项目/成本中心	风险与资产边界
典型产出	可部署的软件包	带签名、 SBOM 的可验证资产

封板的核心在于责任边界的显式定义：通过技术手段（如签名、SBOM）和管理流程（如维护责任确认），将模糊的、持续的维护义务转化为明确的、有限的责任边界。

封板与发布的实践关系：需要明确的是，"发布"与"封板"在技术动作上常有重叠（如打Tag、构建镜像）。两者的核心区别在于治理意图与面向对象。"发布"主要是一个对外的交付动作，面向市场和用户，宣告新功能的可用性；而"封板"本质上是一个对内的治理动作，面向组织内部的运维、审计和财务部门，旨在终止代码的流变状态，锁定内部维护责任与成本。许多软件对外成功"发布"，但对内而言仍是无人敢碰、责任不清的"流变黑洞"。"封板"的价值，正是为了在组织内部建立这道清晰的治理边界。

4.2 "封板"强度谱系与实施指引

表4 封板强度谱系及其实施

Table 4 Sealing intensity spectrum and implementation

强度等级	典型形态	核心承诺	关键技术门槛/实施成本	适用场景
强封板	容器镜像、签名二进制包	环境与行为的完全确定性。	高：需建设镜像仓库、签名体系、完备的供应链安全工具链。	生产部署、商业分发、信创等高合规场景。
中封板	语义化版本包、API服务+SLA	接口兼容性承诺。	中：需制定并维护API契约，建立契约测试与SLA监控。	微服务、开源库、平台服务。

强度等级	典型形态	核心承诺	关键技术门槛/实施成本	适用场景
弱封板	通过CI的Git提交	特定时点状态可复现。	低：需标准化CI流水线与版本标签规范。	高频开发、特性分支。

案例转化路径：银行COBOL模块改造：1) 编制OpenAPI规范并实现契约测试（中封板）；2) 将程序与运行时打包为签名**Docker**镜像，并生成SBOM（强封板），从而产出"利息计算服务"资产。

4.2.1 封板检查清单（工程实践参考）

为提升可操作性，我们总结了强封板的典型检查项，其中经济封板是区别于常规流程的核心：

1. 经济封板：

- ☐ 未来N年预估维护成本已计提
- ☐ 资产健康度基线建立
- ☐ 衰减预警阈值设定

2. 技术封板：

- ☐ 代码版本锁定（Git tag + hash）
- ☐ 构建环境可重现（Dockerfile/构建脚本）
- ☐ 所有依赖明确（SBOM生成）
- ☐ 通过安全扫描（无高危漏洞）

3. 责任封板：

- ☐ API契约文档化（OpenAPI/Swagger）
- ☐ 服务级别目标（SLO）定义
- ☐ 维护责任交接确认
- ☐ 知识文档归档

4.3 "封板衰减"与"解封"：动态治理闭环

封板衰减：资产价值因技术过时（如依赖EOL）或需求变化而衰减，状态由"有效"向"风险预警"乃至"失效"迁移。

解封（Unsealing）与再负债化：为应对紧急修复、微小功能调整或战略重构，对已封板资产进行修改，即触发"解封"。此举使资产重新进入流变状态，维护义务与成本负债即刻恢复（再负债化），并产生待处理的"解封债务"。为防止"解封"滥用导致治理失效，应设立明确的"解封门槛"，例如要求必须经过变更控制委员会（CCB）的审批，并为每笔"解封债务"设定严格的"偿还期限"（如必须在7天内完成"再封板"），确保系统不会长期暴露在"流变"风险中。该术语仅用于描述工程治理状态变化，不涉及安全合规违规判定。建议将此审计与CI/CD流水线集成，实现自动记录与跟踪。

资产替换与升级：治理的更高形态并非永远维护单一资产。当通过"解封-修改-再封板"的成本高于培育一个拥有新能力的新产出物并替换旧资产时，应启动资产替换流程。新资产的封板，标志着旧资产负债的终结与新资产生命周期的开始。

再封板：是对衰减或解封债务的响应，即通过重新构建、测试与部署，使资产回归有效封板状态，完成治理闭环。

案例演进：银行服务镜像因基础OS EOL进入"预警"。团队评估后决定"再封板"（升级基础镜像）。若其间发生过热修复（解封），则需在再封板中一并清偿债务。

4.4 封板强度决策权衡：一个简易框架

为平衡"资产确定性"与"变更敏捷性"，可基于模块的两个关键维度进行封板强度决策：

- 1. 业务稳定性（年功能变更频率，F）
- 2. 故障影响度（失败成本，C）

建议决策逻辑如下：

业务稳定性 (F)	故障影响度 (C)	建议封板强度	治理侧重点
低 (F ≤ 2次/年)	高 (宕机损失大/合规风险高)	强封板	保障确定性与合规
中 (2 < F ≤ 6次/年)	中	中封板 (API契约+SLA)	平衡兼容性与可控性
高 (F > 6次/年)	低	弱封板 (或暂不封板)	保障开发部署敏捷性

注：阈值（2次/年，6次/年）可根据组织具体上下文调整。此框架参考了技术债权衡决策研究中对"变更频率"与"重构成本"的考量[9]。

五、案例全生命周期映射与效益分析

案例研究方法说明：本文的银行案例是一个"复合示范性案例"，其场景基于作者在金融科技领域的多年咨询实践提炼，数据则根据行业公开报告[1]中"大型银行核心系统年均维护成本约占初始投资150%-300%"的典型区间，结合对多位金融科技架构师的访谈，选取中位值进行保守推演生成。关键指标如"维护人力减少"参考了DevOps状态报告（DORA）中对容器化技术提升运维效率的统计范围[14]；"MTTR缩短"则基于不可变部署模式对故障恢复时间的典型改善数据[14]。本案例旨在系统展示理论框架各环节的输入、行动与逻辑输出，而非报告单一项目的审计结果。这种研究方法在提出新治理框架的初期是常见且必要的。

表5 复合示范性案例映射与效益推演
Table 5 Composite demonstration case mapping and benefit derivation
（数据来源：基于行业基准的复合模型推演，非单一项目实测）

理论阶段	银行COBOL模块状态	管理行动与封板策略	典型收益/风险改进指标（逻辑推演）
负债识别	遗产代码，专家退休，文档缺失，年维护成本>100万元。	列入高风险清单，启动资产化项目。	识别出主要成本沉没点；风险量化评分：8.5/9（高风险）
资产转化	逻辑复杂但接口可标准化。	1. API契约化（中封板）；2. 容器化与签名（强封板）。	维护人力大幅减少（参考DORA报告Elite组典型改善幅度）；部署周期从周级缩短至分钟级；年度维护成本显著下降
衰减预警	依赖的OS版本EOL，安全扫描告警。	监控系统预警，启动再封板评估。	提前90天预警，规避高危漏洞风险；合规检查成本：预估20万（若未预警）→5万（提前处理）
解封事件	紧急合规调整，直接修改生产容器。	流水线自动记录"解封债务"。	使隐性技术债显性化：产生1项解封债务，需3人日清偿
债务偿还与再封板	配置漂移增加运维风险。	执行再封板：升级并重新部署。	MTTR显著缩短（参考DORA报告不可变部署模式典型改善）；资产健康度评分恢复至高位

敏感性分析说明：上述效益指标基于DORA《Accelerate》报告中Elite/High performer组的典型改善区间推演。即使在保守假设下（改善幅度下调20%），框架仍能带来显著的成本节约与风险可控性提升，表明本治理框架的效益具备一定的鲁棒性。

权衡与讨论：需要指出，"封板"在提升确定性与降低长期成本的同时，可能引入"敏捷性摩擦"（Agility Friction）：对已封板资产的任何修改都需启动"解封-再封板"流程，增加了紧急响应的开销。因此，封板强度应与模块变更频率、业务关键性相匹配（参见表4及4.4节决策框架）。治理的精髓在于权衡，而非绝对化。

决策场景对比：

- 改造前："是否重构COBOL模块？" → 因责任不清、成本未知，管理层决策"不动，凑合用"（风险隐匿）。
- 改造后："是否对'利息计算服务'资产进行再封板？" → 系统显示：升级成本15人日，预计新增维护负债周期3年，年化成本25万/年 → 管理层基于清晰数据决策（风险显性化）。

六、实践启示与未来方向

6.1 管理启示：从技术到治理

- 度量体系重构：从"代码量"转向"资产健康度"（基于表2）与"封板成熟度"（基于表4）。
- 工程流程再造：CI/CD的核心输出应定义为"可信产出物"，内嵌签名、SBOM生成等强制门禁。
- 治理机制延伸：建立覆盖全生命周期的产出物资产管理制度，并建立内部"技术负债风险附录"。
- 管理层沟通策略：将"封板"诠释为"将不确定的运维成本转化为可控的资产投资"，将技术语言转化为预算与风险管理语言，争取资源支持。

6.2 落地路线图（五步法）

针对希望导入本框架的组织，建议按以下路线分阶段实施：

Step 1：识别高风险模块

- └ 运用表2评估存量代码EML风险
- └ 输出：优先改造清单（按风险/价值排序）

Step 2：选择封板强度

- └ 根据4.4节决策框架，匹配业务稳定性与故障影响度
- └ 输出：模块-封板强度映射表

Step 3：建设流水线门禁

- └ CI/CD集成：版本锁定、SBOM生成、签名、安全扫描
- └ 输出：封板自动化流水线

Step 4：设定衰减预警

- └ 监控依赖EOL、漏洞告警、SLO违约
- └ 输出：资产健康度仪表盘

Step 5：解封审批与再封板SLA

- └ 定义解封门槛（如CCB审批）与偿还期限（如7天内再封板）
- └ 输出：治理闭环与审计记录

6.3 研究局限与未来方向

局限：评估框架的普适性需更多行业验证；"封板衰减"模型有待基于真实数据（如开源项目版本历史）的生存分析深化；对"敏捷性摩擦"的量化管理需进一步研究。

未来方向：

- 智能化负债管理：利用代码分析、仓库活动等数据，构建负债风险预测与智能封板决策模型。
- 软件资产经济估值：探索结合复用度、稳定性、替代成本的产出物经济价值评估框架。

3. 本土化实践集成：研究与国内领先的DevOps平台、信创软件资产管理解决方案的深度融合路径，推动理论落地。

七、结论

银行COBOL核心模块的资产化重生之旅，完整展示了"存在性负债→封板→产出物资产→衰减与再封板"的治理闭环。这一案例表明，软件工程的价值锚点应从"代码生产"转向"资产化交付"。

本文提出的框架——在工程治理意义上，持续演进的代码更接近负债载体，而封板后的产出物才构成可治理资产，不仅为破解软件维护成本困局提供了新的分析工具，更具强烈的现实意义：在中国软件业人员高流动的市场上，它是构建组织级研发韧性的工程基石；在信创国家战略下，它是构筑安全可信软件供应链的治理抓手。推动从"代码本位"到"产出物资产本位"的转型，是产业回归工程本质、迈向高质量发展的理性选择。

需要再次强调的是，本文并非否定软件代码作为无形资产的经济价值，而是揭示：唯有通过"封板"等显性治理手段，剥离其附着的不确定性维护负债，其真正的资产属性才能得以稳固呈现和可靠计量。

（本框架同样适用于制造业、能源、政务等领域的复杂遗留系统现代化与国产化软件资产治理。）

参考文献

- [1] 中国软件行业协会. 中国软件产业发展报告（2023）[R]. 北京：中国软件行业协会, 2023.
- [2] 国家金融监督管理总局. 关于2022年度银行保险机构信息科技风险管理情况的通报[Z]. 2023.
- [3] Cunningham W. The WyCash Portfolio Management System[C]//OOPSLA'92 Experience Report. 1992.
- [4] 中华人民共和国财政部. 企业会计准则第13号——或有事项[S]. 2006.
- [5] Eghbal N. Working in Public: The Making and Maintenance of Open Source Software[M]. Stripe Press, 2020.
- [6] Bader M, Chintamaneni P K. Software Bill of Materials: Survey and Perspectives[J]. IEEE Security & Privacy, 2022, 20(4): 86–93. DOI: 10.1109/MSEC.2022.3141690
- [7] Google. SLSA: Supply-chain Levels for Software Artifacts v1.0[EB/OL]. <https://slsa.dev/spec/v1.0>, 2023.
- [8] Li Z, Avgeriou P, Liang P. A systematic mapping study on technical debt and its management[J]. Journal of Systems and Software, 2015, 101: 193–220.
- [9] Besker T, Martini A, Lokuge R E, et al. Embracing technical debt, from a startup company perspective[C]//2018 IEEE International Conference on Software Maintenance and Evolution (ICSME). IEEE, 2018: 415–425.
- [10] Xiao L, Cai Y, Kazman R. Design rule spaces: a new form of architecture insight[C]//Proceedings of the 36th International Conference on Software

Engineering. 2014: 967-977.

[11] 全国信息安全标准化技术委员会. 网络安全标准实践指南—软件供应链安全管理要求（正式稿）[Z]. 2024.

[12] Boehm, B. W., & Papaccio, P. N. (1988). Understanding and controlling software costs. *IEEE Transactions on Software Engineering*, 14(10), 1462-1477.

[13] Ampatzoglou, A., et al. (2019). The financial aspect of managing technical debt: A systematic literature review. *Information and Software Technology*, 114, 1-18.

[14] Forsgren N, Humble J, Kim G. Accelerate: The Science of Lean Software and DevOps[M]. IT Revolution Press, 2018.

致谢：感谢审稿专家及主编提出的宝贵意见。

作者简介：付毅（1973—），男，无职称，无学位，川北医学院后勤管理处。E-mail: fuyi@nsmc.edu.cn

Biography: FU Yi (1973—), male, no professional title, no degree, Logistics Management Office, North Sichuan Medical College. E-mail: fuyi@nsmc.edu.cn

通讯作者/**Corresponding author:** 付毅, E-mail: fuyi@nsmc.edu.cn

联系电话/**Tel:** 17781158558

传真/**Fax:** 无 / None

基金项目/**Funding:** 无 / None

地址/**Address:** 四川省南充市顺庆区大学城, 637000 / University Town, Shunqing District, Nanchong, Sichuan 637000, China

1. 本文"负债"均指管理会计语境, 非《企业会计准则》下的法定负债。↵